LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression

Zhuoshi Pan[1]†, Qianhui Wu[2]‡, Huiqiang Jiang[2], Menglin Xia[2], Xufang Luo[2], Jue Zhang[2], Qingwei Lin[2], Victor Rühle[2], Yuqing Yang[2], Chin-Yew Lin[2], H. Vicky Zhao[1], Lili Qiu[2], Dongmei Zhang[2]

[1] Tsinghua University, [2] Microsoft Corporation

{qianhuiwu, hjiang, xufang.luo}@microsoft.com

arXiv:2403.12968v2 [cs.CL] 12 Aug 2024

Abstract

This paper focuses on task-agnostic prompt compression for better generalizability and efficiency. Considering the redundancy in natural language, existing approaches compress prompts by removing tokens or lexical units according to their information entropy obtained from a causal language model such as LLaMa-7B. The challenge is that information entropy may be a suboptimal compression metric: (i) it only leverages unidirectional context and may fail to capture all essential information needed for prompt compression; (ii) it is not aligned with the prompt compression objective.

To address these issues, we propose a data distillation procedure to derive knowledge from an LLM to compress prompts without losing crucial information, and meantime, introduce an extractive text compression dataset. We formulate prompt compression as a token classification problem to guarantee the faithfulness of the compressed prompt to the original one, and use a Transformer encoder as the base architecture to capture all essential information for prompt compression from the full bidirectional context. Our approach leads to lower latency by explicitly learning the compression objective with smaller models such as XLM-RoBERTa-large and mBERT.

We evaluate our method on both in-domain and out-of-domain datasets, including MeetingBank, LongBench, ZeroScrolls, GSM8K, and BBH. Despite its small size, our model shows significant performance gains over strong baselines and demonstrates robust generalization ability across different LLMs. Additionally, our model is 3x-6x faster than existing prompt compression methods, while accelerating the end-to-end latency by 1.6x-2.9x with compression ratios of 2x-5x.[1]

1 Introduction

Recent years have witnessed the emergence of various prompting techniques for large language models (LLMs), such as Chain-of-Thought (COT) (Wei et al., 2022), In-context Learning (ICL) (Dong et al., 2023), and Retrieval Augmented Generation (RAG) (Lewis et al., 2020). These techniques empower LLMs to handle complex and varied tasks through rich and informative prompts that may exceed tens of thousands of tokens. However, the benefits of such lengthy prompts come at a cost of increased computational and financial overhead, as well as the degraded information perception ability of LLMs. Prompt compression is a straightforward solution to address these issues, which attempts to *shorten the original prompts without losing essential information*.

Several methods have been proposed to compress prompts in a *task-aware* manner (Jiang et al., 2023b; Xu et al., 2024; Jung and Kim, 2023; Huang et al., 2023). These techniques aim to generate compressed prompts tailored to the specific task or query, typically resulting in enhanced performance on downstream tasks, particularly in question answering. However, the dependency on task-specific features presents challenges in terms of efficiency and generalizability when deploying these methods. For example, in RAG-style applications, it may become necessary to compress the same documents multiple times depending on the associated queries with task-aware prompt compression. More details are discussed in Sec. 2.

Some works have explored *task-agnostic* prompt compression methods for better generalizability and efficiency (Jiang et al., 2023a; Li et al., 2023). The underlying assumption is that *natural language contains redundancy (Shannon, 1951) that may be useful for human understanding but might not be necessary for LLMs*. Therefore, they propose to compress prompts by removing tokens (Jiang et al.,

†Work during internship at Microsoft.

‡Corresponding author.

[1]Code: https://aka.ms/LLMLingua-2

2023a) or lexical units (Li et al., 2023) according to their information entropy obtained from a causal small language model (SLM), regardless of the downstream task or question information. However, these task-agnostic methods face two challenges: (i) Information entropy is an empirical metric for prompt compression. Relying on it for prompt trimming may be suboptimal, as it is not aligned with the prompt compression objective. (ii) Causal LMs only leverage unidirectional context, which may fail to capture all essential information needed for prompt compression within the context.

The challenges lead to the following research questions:

Q1. How can we identify or build a suitable dataset to align the SLM towards effective prompt compression?

Q2. How can we design a compression algorithm that effectively leverages the full bidirectional context for better performance?

For Q1, most text compression datasets are *abstractive* (Toutanova et al., 2016; Koupaee and Wang, 2018; Kim et al., 2019), meaning that they treat prompt compression as a generative task where the original prompts are rephrased into condensed ones. However, this autoregressive generation process is slow and it may produce hallucinated content (Zhao et al., 2020). On the other hand, *extractive* compression datasets such as SentComp (Filippova and Altun, 2013) and DebateSum (Roush and Balaji, 2020) are usually created for the summarization task and often lack detailed information. In the case of prompt compression, this will hurt the performance of LLM inference in downstream applications such as QA (see Appendix G for some examples). Therefore, it is necessary to construct an extractive text compression dataset that retains essential information.

Contributions. We present this paper to address the above challenges for task-agnostic prompt compression. We make the following contributions.

- We propose a data distillation procedure to derive knowledge from an LLM (GPT-4) to compress the prompts without losing crucial information. We introduce an extractive text compression dataset, containing pairs of original texts from MeetingBank (Hu et al., 2023) and their compressed versions. We publicly release the dataset.
- We approach prompt compression as a token classification task (*i.e.*, preserve or discard), and take the predicted probability of each token being labeled as `preserve` as the compression metric. The benefits are three folds: (1) It can capture all essential information needed for prompt compression from the full bidirectional context by using a Transformer encoder for feature extraction. (2) It can lead to lower latency, due to the use of smaller models to explicitly learn the compression objective. (3) It guarantees faithfulness of the compressed prompt to the original content.
- We conduct extensive experiments and analysis on both in-domain (*i.e.*, MeetingBank) and out-of-domain datasets (*i.e.*, LongBench, ZeroScrolls, GSM8K, and Big Bench Hard). Despite small in size, our model shows significant performance gains over strong baselines and demonstrates robust generalization ability from GPT-3.5-Turbo to Mistral-7B. Additionally, our model is 3x-6x faster than existing prompt compression methods, while accelerating the end-to-end latency by 1.6x-2.9x with compression ratios of 2x-5x.

2 Related Works

Depending on whether task information is used for compression, prompt compression methods can be categorized into task-aware and task-agnostic compression approaches.

Task-aware compression compresses the context based on the downstream task or the current query. For example, LongLLMLingua (Jiang et al., 2023b) applies a question-aware coarse-to-fine compression approach to estimate the information entropy of the tokens and adapts the estimation according to the question. Reinforcement Learning (RL) based methods (Jung and Kim, 2023; Huang et al., 2023) usually train a model for prompt compression with reward signals from downstream tasks. Soft prompt tuning methods (Wingate et al., 2022; Mu et al., 2023) typically require fine-tuning for the specific task. Xu et al. (2024) trains a summarization model to compress the context depending on the question. Task-aware compression approaches are usually tailored for specific tasks and compression ratios, which may limit their generalizability in real-world applications.

Task-agnostic methods compress the prompt without considering the specific task, making it

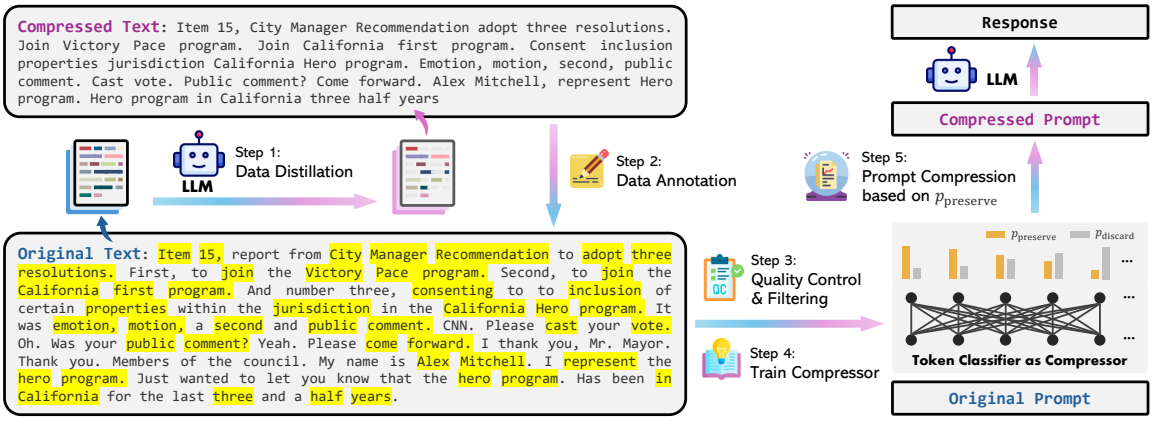


Figure 1: Overview of LLMLingua-2.

more adaptable to a range of applications and black-box LLMs. However, producing compressed text that can generalize well to different tasks is not trivial. Typical methods involve using information entropy-based metrics to remove redundant information in the prompt (Li et al., 2023; Jiang et al., 2023a). They employ a small language model to estimate token importance from the information metrics. Despite being training-free, these methods may not effectively capture the token importance distribution optimized for specific LLMs and often entail high computation overhead. Summarization-based methods are also leveraged for task-agnostic compression (Chen et al., 2023; Packer et al., 2023). However, they often omit crucial details and do not generalize well. An alternative approach is to compress or trim the context hidden or KV caches (Chevalier et al., 2023; Ge et al., 2024; Zhang et al., 2023; Liu et al., 2023; Xiao et al., 2024). However, this is orthogonal to our work and cannot be easily applied to black-box LLMs.

3 Dataset Construction

In this section, we outline the process of dataset construction for prompt compression. We first introduce our data distillation procedure, which involves extracting knowledge from an LLM (GPT-4) to compress texts without losing crucial information or introducing hallucinated content (Sec. 3.1). Leveraging the distilled knowledge from the LLM, we explain our data annotation algorithm, which assigns labels to each word in the original text to indicate whether it should be preserved after compression (Sec. 3.2). To ensure the dataset's quality, we propose two quality control metrics for filtering low-quality samples (Sec. 3.3).

3.1 Data Distillation

To extract knowledge from the LLM for effective prompt compression, our goal is to prompt GPT-4 to generate compressed texts from original texts that meet the following criteria: (i) *Token reduction*: Compressed prompts should be short in length to reduce cost and speed up inference. (ii) *Informativeness*: Essential information should be retained. (iii) *Faithfulness*: Compressed prompts should remain faithful and avoid introducing hallucinated content to ensure accuracy when prompting LLMs in downstream tasks.

However, distilling such data from GPT-4 is challenging, as it does not consistently follow the instructions. For instance, Jiang et al. (2023a) experimented with different prompts for compression and found that GPT-4 struggles to retain essential information from original texts. In our preliminary experiments, we have also observed that GPT-4 tends to modify expressions used in the original texts and sometimes generates hallucinated content. To address this challenge, we propose the following dataset distillation procedure.

Instruction Design A well-crafted instruction is the key to unveiling the compression capabilities of GPT-4. To ensure that the generated texts stay *faithful* to the original, we explicitly instruct GPT-4 to compress the text by discarding unimportant words in the original texts only and not adding any new words during generation.

To ensure *token reduction* and *informativeness*, previous studies (Jiang et al., 2023a; Huang et al., 2023) have specified either a compression ratio or a target number of compressed tokens in the instructions. However, GPT-4 often fails to adhere to these restrictions. Additionally, the information

Our Instruction for Compression:
Compress the given text to short expressions, and such that you (GPT-4) can reconstruct it as close as possible to the original. Unlike the usual text compression, I need you to comply with the 5 conditions below:
1. You can ONLY remove unimportant words.
2. Do not reorder the original words.
3. Do not change the original words.
4. Do not use abbreviations or emojis.
5. Do not add new words or symbols.
Compress the origin aggressively by removing words only. Compress the origin as short as you can, while retaining as much information as possible. If you understand, please compress the following text: {*text to compress*}
The compressed text is:

Figure 2: Our instruction used for data distillation.

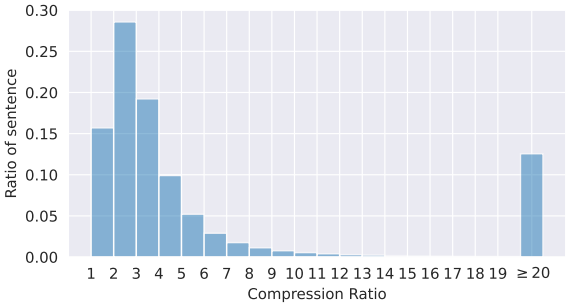


Figure 3: Distribution of compression ratio after chunk-wise compression on MeetingBank.

density of text can vary significantly depending on its genre, style, etc. For instance, news articles typically contain denser information compared to meeting transcripts. Furthermore, even within the domain of meeting transcripts, the information density from different speakers may vary. These factors suggest that a fixed compression ratio may not be optimal. Therefore, we remove the compression ratio restriction from our instructions and instead prompt GPT-4 to compress the origin text as short as possible while retaining as much information as possible. As shown in Fig. 3, GPT-4 assigns varying compression ratios to different sentences and discards some sentences entirely. For a comparison between our instruction and those of Jiang et al. (2023a), please refer to Table 7.

Chunk-Wise Compression Empirically, we have found that the length of the original text has a notable influence on the compression performance. As shown in Fig. 4, GPT-4 tends to apply a high compression ratio when processing very long context, which might be due to GPT-4's limited ability to handle long context. This aggressive compression leads to substantial information loss, significantly impacting the performance of downstream tasks. To mitigate this issue, we first segment each long context into multiple chunks, each containing no more than 512 tokens and ending with a period. We then instruct GPT-4 to compress each chunk individually.

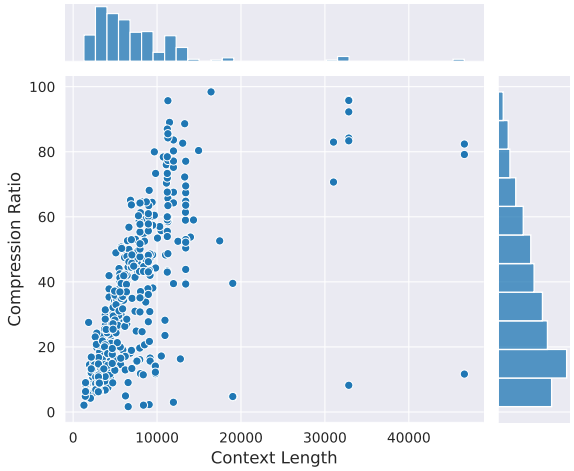


Figure 4: Illustration of compression ratio *w.r.t.* original context length on MeetingBank. We use GPT-4-32k with the output token limit setting to 4096.

3.2 Data Annotation

Having obtained pairs of original texts and their compressed versions from data distillation (Sec. 3.1), the goal of data annotation is to assign a *binary* label to each token in the original texts to determine if it should be preserved or discarded after compression. Fig. 5 describes the three primary obstacles encountered here, which arise from GPT-4's inability to precisely comply with the instruction in Fig. 9. Alg. 1 outlines the overall procedure of the proposed annotation algorithm designed to deal with these obstacles. For more detailed information, please refer to Appendix B.

3.3 Quality Control

We introduce two quality control metrics to assess the quality of the compressed texts generated by GPT-4 distillation, as well as the quality of the automatically annotated labels. We then filter the examples by their scores.

Variation Rate As GPT-4 may fail to follow the instructions, we introduce the metric *Variation Rate (VR)* to evaluate the quality of the compressed texts generated from data distillation. VR measures the proportion of words in the compressed text that are

Original Texts
Item 15, report from City Manager Recommendation to adopt three resolutions. First, to join the Victory Pace program. Second, to join the California first program. And number three, consenting to to inclusion of certain properties within the jurisdiction in the California Hero program.

Compressed Texts
City Manager Recommendation adopt three resolutions. Join California first program. Consent properties inclusion jurisdiction California Hero program.

Figure 5: Challenges in data annotation.
(i) Ambiguity: a word in the compressed texts may appear multiple times in the original content.
(ii) Variation: GPT-4 may modify the original words in tense, plural form, *etc.* during compression.
(iii) Reordering: The order of words may be changed after compression.

Algorithm 1: Data Annotation

Input : original string S_{ori}, compressed string S_{comp}, window size s.
Split original string S_{ori} to word list $\mathbb{S}_{ori}$.
Split compressed S_{comp} to word list $\mathbb{S}_{comp}$.
Initialize labels of original words to *False*.
Initialize previous match index $prev$ to 0.
for $w \in \mathbb{S}_{comp}$ **do**
  for $i = 1, 2, ..., \frac{s}{2}$ **do**
    $right = \min(|\mathbb{S}_{ori}|, prev + i)$
    if *fuzzy_match(w,* $\mathbb{S}_{ori}[right]$*)* **then**
      $\mathbb{L}[right]$ = *True*.
      $prev = right$.
      Break.
    end
    $left = \max(0, prev - i)$
    if *fuzzy_match(w,* $\mathbb{S}_{ori}[left]$*)* **then**
      $\mathbb{L}[left]$ = *True*.
      Break.
    end
  end
end
Output : labels of original words $\mathbb{L}(\mathbb{S}_{ori})$.

absent in the original text. Specifically, let $\mathbb{S}_{comp}$ be the set of words in the compressed text and $\mathbb{S}_{ori}$ be that of the original text. VR is defined as:

$$VR = \frac{1}{|\mathbb{S}_{comp}|} \sum_{w \in \mathbb{S}_{comp}} \mathbb{I}(w \notin \mathbb{S}_{ori}), \quad (1)$$

where $|\cdot|$ is the cardinality of a set. A higher variation rate implies a higher likelihood of encountering hallucinated content. Therefore, we exclude the examples with the top 5% highest variation rates.

Alignment Gap We propose *Alignment Gap (AG)* to evaluate the quality of the automatically annotated labels. Let $l(\cdot)$ represent the annotation function, where $l(w) = True$ signifies that word $w \in \mathbb{S}_{ori}$ corresponds to a word in $\mathbb{S}_{comp}$. We firstly define the matching rate (MR) as:

$$MR = \frac{1}{|\mathbb{S}_{ori}|} \sum_{w \in \mathbb{S}_{ori}} \mathbb{I}(l(w) = True). \quad (2)$$

Since there exists a many-to-one word mapping from $\mathbb{S}_{ori}$ to $\mathbb{S}_{comp}$ (*i.e.*, the "Ambiguity" challenge presented in Sec. 3.2), we further present a hitting rate (HR) as a regularization term to measure the proportion of words in $\mathbb{S}_{comp}$ that are found in $\mathbb{S}_{ori}$. HR is defined as:

$$HR = \frac{1}{|\mathbb{S}_{ori}|} \sum_{w \in \mathbb{S}_{comp}} \mathbb{I}(w \in \mathbb{S}_{ori}). \quad (3)$$

Finally, the Alignment Gap (AG) is defined as:

$$AG = HR - MR. \quad (4)$$

The alignment gap of a perfect annotation should be 0. A large AG indicates a high hitting rate with a poor matching rate, implying low-quality annotation for this example. Therefore, we discard examples of the highest 10% alignment gap to ensure quality control of the dataset.

4 Compressor

We formulate prompt compression as a binary token classification problem (*i.e.*, preserve or discard) to guarantee the faithfulness of the compressed prompt to the original content, and meantime ensure the low latency of the compression model itself. For the token classification model, we employ a Transformer encoder as the feature extractor to leverage information from the bidirectional contexts of each token. We train the classification model on the dataset constructed in Sec. 3 from MeetingBank (Hu et al., 2023). During inference, we determine whether to preserve or discard each token in the original prompt based on its probability calculated by our classification model.

4.1 Token Classification Model

Architecture We utilize a Transformer encoder (Devlin et al., 2019) as the feature encoder f_θ and add a linear classification layer on top. Given

Methods	QA	Summary					Length	
	EM	BLEU	Rouge1	Rouge2	RougeL	BERTScore	Tokens	$1/\tau$
Selective-Context	66.28	10.83	39.21	18.73	27.67	84.48	1,222	2.5x
LLMLingua	67.52	8.94	37.98	14.08	26.58	86.42	1,176	2.5x
LLMLingua-2-small	85.82	**17.41**	48.33	**23.07**	**34.36**	**88.77**	984	3.0x
LLMLingua-2	**86.92**	17.37	**48.64**	22.96	34.24	88.27	970	3.1x
Original	87.75	22.34	47.28	26.66	35.15	88.96	3,003	1.0x

Table 1: In-domain evaluation of different methods on MeetingBank.

an original prompt consisting of N words $\boldsymbol{x} = \{x_i\}_{i=1}^N$, this can be formulated as:

$$\boldsymbol{h} = f_\theta(\boldsymbol{x}), \tag{5}$$

$$p(x_i, \Theta) = \text{softmax}(Wh_i + b), \tag{6}$$

where $\boldsymbol{h} = \{h_i\}_{i=1}^N$ denotes feature vectors for all words, $p(x_i, \Theta) \in \mathbb{R}^2$ denotes the probability distribution of labels $\{\texttt{preserve}, \texttt{discard}\}$ for the i-th word x_i, and $\Theta = \{\theta, W, b\}$ represent all the trainable parameters.

Training Let $\boldsymbol{y} = \{y_i\}_{i=1}^N$ denote the corresponding labels for all words in $\boldsymbol{x}$, then we employ cross entropy loss to train the model. The loss function $\mathcal{L}$ *w.r.t.* $\boldsymbol{x}$ is:

$$\mathcal{L}(\Theta) = \frac{1}{N} \sum_{i=1}^{N} \text{CrossEntropy}(y_i, p(x_i, \Theta)). \tag{7}$$

4.2 Compression Strategy

Our approach to compressing the original prompt $\boldsymbol{x} = \{x_i\}_{i=1}^N$ with a target compression ratio $1/\tau$ involves a three-step process, where τ is defined as the quotient of the number of words in the compressed prompt and the number of words in the original prompt $\boldsymbol{x}$. First, we derive the target number of tokens to be preserved in the compressed prompt $\tilde{\boldsymbol{x}}$: $\tilde{N} = \tau N$. Next, we use the token classification model to predict the probability p_i of each word x_i being labeled as `preserve`[2]. Finally, we retain the top $\tilde{N}$ words in the original prompt $\boldsymbol{x}$ with the highest p_i and maintain their original order to form the compressed prompt $\tilde{\boldsymbol{x}}$.

It's worth noting that our approach can be readily integrated into the coarse-to-fine framework proposed in LLMLingua (Jiang et al., 2023a), allowing for a higher compression ratio of ∼15x for tasks involving multiple demonstrations or documents. Particularly, we can replace the perplexity-based iterative token compression module in LLMLingua with our token-classification-based compressor, while keeping the budget controller unchanged. Detailed information can be found in Appendix K.

[2]To address tokenization-related challenges that arise when applying our approach across various LLMs and SLMs, we preserve the integrity of multi-token words and represent the probability of a word by averaging over the predicted probabilities of all subword tokens.

5 Experiment

Implementation Details We construct our extractive text compression dataset using training examples from MeetingBank (Hu et al., 2023) with implementation details in Appendix A. Our approach is implemented using Huggingface's Transformers and PyTorch 2.0.1 with CUDA-11.7. We use `xlm-roberta-large` (Conneau et al., 2020) and `multilingual-BERT` (Devlin et al., 2019) for the feature encoder f_θ in our compressor, which we refer to as `LLMLingua-2` and `LLMLingua-2-small`, respectively. We fine-tune both models for 10 epochs, using the Adam optimizer (Kingma and Ba, 2015) with a learning rate of 1e-5 and a batch size of 10. Unless specified otherwise, all reported metrics use GPT-3.5-Turbo-0613[3] as the target LLM for downstream tasks, with greedy decoding at a temperature of 0 for enhanced stability across experiments.

Datasets & Evaluation Metrics We conduct five groups of experiments to evaluate the compressed prompts on two groups of datasets.

(i) In-Domain: As we train our compressor using the dataset built with training examples from MeetingBank (Hu et al., 2023), we use the **MeetingBank** test examples for in-domain evaluation. In addition to the *summarization* task, we further introduce a *QA* task by prompting GPT-4 to generate 3 question-answer pairs for each example distributed across the whole context (see Appendix F

[3]https://platform.openai.com/

Methods	LongBench									ZeroSCROLLS		
	SingleDoc	MultiDoc	Summ.	FewShot	Synth.	Code	**AVG**	Tokens	$1/\tau$	**AVG**	Tokens	$1/\tau$
2,000-token constraint												
Task(Question)-Aware Compression												
SBERT†	33.8	35.9	25.9	23.5	18.0	17.8	25.8	1,947	5x	20.5	1,773	6x
OpenAI†	34.3	36.3	24.7	32.4	26.3	24.8	29.8	1,991	5x	20.6	1,784	5x
LongLLMLingua†	39.0	42.2	27.4	69.3	53.8	56.6	48.0	1,809	6x	32.5	1,753	6x
Task(Question)-Agnostic Compression												
Selective-Context†	16.2	**34.8**	24.4	15.7	8.4	49.2	24.8	1,925	5x	19.4	1,865	5x
LLMLingua†	22.4	32.1	24.5	61.2	10.4	56.8	34.6	1,950	5x	27.2	1,862	5x
LLMLingua-2-small	29.5	32.0	24.5	64.8	**22.3**	56.2	38.2	1,891	5x	33.3	1,862	5x
LLMLingua-2	**29.8**	33.1	**25.3**	**66.4**	21.3	**58.9**	**39.1**	1,954	5x	**33.4**	1,898	5x
3,000-tokens constraint												
Task(Question)-Aware Compression												
SBERT†	35.3	37.4	26.7	63.4	51.0	34.5	41.4	3,399	3x	24.0	3,340	3x
OpenAI†	34.5	38.6	26.8	63.4	49.6	37.6	41.7	3,421	3x	22.4	3,362	3x
LongLLMLingua†	40.7	46.2	27.2	70.6	53.0	55.2	48.8	3,283	3x	32.8	3,412	3x
Task(Question)-Agnostic Compression												
Selective-Context†	23.3	**39.2**	25.0	23.8	**27.5**	53.1	32.0	3,328	3x	20.7	3,460	3x
LLMLingua†	31.8	37.5	26.2	67.2	8.3	53.2	37.4	3,421	3x	30.7	3,366	3x
LLMLingua-2-small	**35.5**	38.1	26.2	67.5	23.9	60.0	41.9	3,278	3x	33.4	3,089	3x
LLMLingua-2	**35.5**	38.7	**26.3**	**69.6**	21.4	**62.8**	**42.4**	3,392	3x	**33.5**	3,206	3x
Original Prompt	39.7	38.7	26.5	67.0	37.8	54.2	44.0	10,295	-	34.7	9,788	-
Zero-Shot	15.6	31.3	15.6	40.7	1.6	36.2	23.5	214	48x	10.8	32	306x

Table 2: Out-of-domain evaluation on general long-context scenarios. †: numbers reported in Jiang et al. (2023b).

Methods	GSM8K						BBH					
	1-shot constraint			half-shot constraint			1-shot constraint			half-shot constraint		
	EM	Tokens	$1/\tau$	EM	Tokens	$1/\tau$	EM	Tokens	$1/\tau$	EM	Tokens	$1/\tau$
Selective-Context†	53.98	452	5x	52.99	218	11x	54.27	276	3x	54.02	155	5x
LLMLingua†	**79.08**	446	5x	77.41	171	14x	**70.11**	288	3x	61.60	171	5x
LLMLingua-2-small	78.92	437	5x	77.48	161	14x	69.54	263	3x	60.35	172	5x
LLMLingua-2	**79.08**	457	5x	**77.79**	178	14x	70.02	269	3x	**61.94**	176	5x
Full-Shot	78.85	2,366	-	78.85	2,366	-	70.07	774	-	70.07	774	-
Zero-Shot	48.75	11	215x	48.75	11	215x	32.32	16	48x	32.32	16	48x

Table 3: Out-of-domain evaluation on reasoning and in-context learning. †: numbers reported in Jiang et al. (2023b).

for more details). For the summarization task, we use the same evaluation metric as in LLMLingua (Jiang et al., 2023a). For QA task, we utilize the Exact Match as the evaluation metric.

(ii) Out-of-Domain: For long-context scenarios, we use **LongBench** (Bai et al., 2023) and **ZeroSCROLLS** (Shaham et al., 2023), and we employ the same evaluation metric as in LongLLMLingua (Jiang et al., 2023b). For reasoning and in-context learning, we use **GSM8K** (Cobbe et al., 2021) and **Big Bench Hard (BBH)** (bench authors, 2023), with evaluation metrics consistent with LLMLingua (Jiang et al., 2023a).

Baselines We take two state-of-the-art prompt compression methods as primary baselines for comparison: Selective-Context (Li et al., 2023) and LLMLingua (Jiang et al., 2023a), both are based on `LLaMA-2-7B`. Additionally, we compare our approach with some task-aware prompt compression methods, such as retrieval-based methods and LongLLMLingua (Jiang et al., 2023b).

Results on In-Domain Benchmark In Table 1, we first present the results of our proposed method

Methods	MeetingBank				LongBench-SingleDoc					
	QA	Summ.	Tokens	$1/\tau$	2,000-token cons.	Tokens	$1/\tau$	3,000-token cons.	Tokens	$1/\tau$
Selective-Context	58.13	26.84	1,222	2.5x	22.0	2,038	7.1x	26.0	3,075	4.7x
LLMLingua	50.45	23.63	1,176	2.5x	19.5	2,054	7.1x	20.8	3,076	4.7x
LLMLingua-2-small	75.97	29.93	984	3.0x	25.3	1,949	7.4x	**27.9**	2,888	5.0x
LLMLingua-2	**76.22**	**30.18**	970	3.0x	**26.8**	1,967	7.4x	27.3	2,853	5.1x
Original Prompt	66.95	26.26	3,003	-	24.5	14,511	-	24.5	14,511	-

Table 4: Evaluation with Mistral-7B as the Target LLM on MeetingBank and LongBench single doc QA task. We report Rouge1(Lin, 2004) for summary.

compared to the strong baselines on MeetingBank. Despite the fact that our compressors are much smaller than the LLaMa-2-7B used in the baselines, our approach achieves significantly better performance on both the QA and Summary tasks, and comes close to matching the performance of the original prompt. This demonstrates the effectiveness of our constructed dataset, and highlights the importance and benefit of optimizing the compression model using prompt compression knowledge.

Results on Out-of-Domain Benchmarks As our model is trained on meeting transcripts data from MeetingBank, here we explore its generalization ability across various benchmarks of long-context scenarios, reasoning, and in-context learning. Table 2 and 3 show the results on LongBench, ZeroSCROLLS, GSM8K, and BBH: Our model has demonstrated superior performance compared to other task-agnostic baselines. Even our smaller model, which is of `BERT-base` size, has been able to achieve comparable, and in some cases, even slightly higher performance than the original prompt. While our approach has shown promising results, it falls short when compared to other task-aware compression methods like LongLLMlingua (Jiang et al., 2023a) on Longbench. We attribute this performance gap to the additional information that they leverage from the question. However, the task-agnostic characteristics of our model make it an efficient option with good generalizability when deployed across different scenarios.

Mistral-7B as the Target LLM Table 4 presents the results of different methods using Mistral-7B-v0.1[4] as the target LLM. Our method demonstrates significant performance gain over other baselines, showcasing its good generalization ability across target LLMs. Notably, LLMLingua-2 yields even better performance than the original prompt. We speculate that Mistral-7B might be less adept at managing long contexts than GPT-3.5-Turbo. Our method, by offering shorter prompts with higher information density, effectively improves Mistral-7B's final inference performance.

[4]https://mistral.ai/

Latency Evaluation Table 5 shows the latency of different systems on a V100-32G GPU with different compression ratios. It shows that LLMLingua-2 has a much smaller computation overhead than other compression methods, and can achieve an end-to-end speedup ranging from 1.6x to 2.9x. Additionally, our method can reduce GPU memory costs by 8x, lowering the demand for hardware resources. For details, see the Appendix I.

$1/\tau$	1x	2x	3x	5x
End2End w/o Compression	14.9			
End2End w/ LLMLingua-2	-	9.4 (1.6x)	7.5 (2.1x)	5.2 (2.9x)
Selective-Context	-	15.9	15.6	15.5
LLMLingua	-	2.9	2.1	1.5
LLMLingua-2	-	**0.5**	**0.4**	**0.4**

Table 5: Latency (s) comparison on MeetingBank.

Observation on Context Awareness We have observed that LLMLingua-2 can effectively maintain the most informative words with respect to the full context as the compression ratio increases. We owe this to the adoption of the bidirectional context-aware feature extractor, as well as the strategy of explicitly optimizing toward the prompt compression objective. See Figure 6 for more details.

Prompt Reconstruction We have conducted experiments of prompting GPT-4 to reconstruct the original prompt from the LLMLingua-2 compressed prompt. The results show that GPT-4 can effectively reconstruct the original prompt, suggesting that there is no essential information loss during the compression process of LLMLingua-2. Figure 7 and 8 in Appendix E present some examples.

Methods	LongBench									ZeroSCROLLS		
	SingleDoc	MultiDoc	Summ.	FewShot	Synth.	Code	**AVG**	Tokens	$1/\tau$	**AVG**	Tokens	$1/\tau$
LLMLingua-2-small	29.5	32.0	24.5	64.8	22.3	56.2	38.2	1,891	5x	33.3	1,862	5x
LLMLingua-2	29.8	33.1	25.3	66.4	21.3	**58.9**	39.1	1,954	5x	**33.4**	1,898	5x
LLMLingua-2‡	**30.7**	**33.9**	**25.4**	**66.6**	**22.6**	58.1	**39.5**	1,853	5x	**33.4**	1,897	5x
Original Prompt	39.7	38.7	26.5	67.0	37.8	54.2	44.0	10,295	-	34.7	9,788	-
Zero-Shot	15.6	31.3	15.6	40.7	1.6	36.2	23.5	214	48x	10.8	32	306x

Table 6: Out-of-domain evaluation on general long-context benchmarks with the 2,000-token constraint. LLMLingua-2‡: We expand the constructed text compression dataset using 50k examples from TriviaQA-wiki. Then train an LLMLingua-2 compressor with the expanded dataset.

Instruction	$1/\tau$	VR $\downarrow$	QA F1 $\uparrow$
Instruction1	123x	13.7	19.1
Instruction2	27x	7.8	26.1
Instruction3	78x	9.6	23.7
Instruction4	49x	9.4	24.9
LLMLingua-2 w/o Chunk	21x	6.0	27.9
LLMLingua-2	2.6x	**2.2**	**36.7**

Table 7: Ablation Study on Chunk-Wise Compression and Instruction Design. We report the compression ratio, variation rate, and QA performance on LongBench Single Document QA. See Fig. 10 in Appendix for more details of Instruction1 - Instruction4 here.

Ablation Study on Chunk-Wise Compression and Instruction Design Table 7 shows that both the designed instruction and the chunk-wise compression strategy proposed in this paper significantly contribute to the success of LLMLingua-2.

6 Conclusion

This paper targets task-agnostic prompt compression for better generalizability and efficiency. In this paper, we identify the challenges encountered in existing methods and address them accordingly. We conduct extensive experiments and analysis on five benchmarks across different tasks and domains. Our model shows superiority over strong baselines in terms of performance and compression latency. We publicly release the dataset of text compression with no essential information loss in this paper.

Limitations

Our text compression dataset was constructed using only training examples from MeetingBank, a dataset of summarization over meeting transcripts. This raises concerns about the generalization ability of our compressor. Here we discuss this question from two perspectives.

Firstly, we have conducted extensive out-of-domain evaluation on four benchmarks in the paper, including LongBench (Bai et al., 2023), ZeroSCROLLS (Shaham et al., 2023), GSM8K (Cobbe et al., 2021), and Big Bench Hard (BBH) (bench authors, 2023), which cover multiple tasks from document QA to math problems and in-context learning. The experimental results show that even our LLMLingua-2-small model that is of `BERT-base` size achieves superior performance than the two LLaMA-2-7B based baselines Selective-Context (Li et al., 2023) and LLMLingua (Jiang et al., 2023a). This demonstrates that our learned prompt compression model has good generalization ability to data from different domains.

Secondly, we expand the constructed text compression dataset using 50k examples from TriviaQA-wiki. Then train an LLMLingua-2 compressor with the expanded dataset to see whether there would be further performance gain. Table 6 shows the results under the 2,000-token constraint. We can see that training the compressor with more data does bring further performance gain (LLMLingua-2‡). However, the improvement seems not that significant. We conjecture that this is because although the semantics of texts from different domains may vary a lot, their redundancy pattern might be similar. Such pattern or knowledge may be learned during in-domain training, and then act as an anchor that can transfer across different domains. We leave this for future work.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. *ArXiv preprint*, abs/2308.14508.

BIG bench authors. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*.

Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. 2023. Walking down the memory maze: Beyond context limit through interactive reading. *ArXiv preprint*, abs/2310.05029.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. *ArXiv preprint*, abs/2305.14788.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *ArXiv preprint*, abs/2110.14168.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. 2023. A survey for in-context learning. *ArXiv preprint*, abs/2301.00234.

Katja Filippova and Yasemin Altun. 2013. Overcoming the lack of parallel data in sentence compression. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1481–1491, Seattle, Washington, USA. Association for Computational Linguistics.

Tao Ge, Hu Jing, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context autoencoder for context compression in a large language model. In *The Twelfth International Conference on Learning Representations*.

Yebowen Hu, Tim Ganter, Hanieh Deilamsalehy, Franck Dernoncourt, Hassan Foroosh, and Fei Liu. 2023. Meetingbank: A benchmark dataset for meeting summarization. *ArXiv preprint*, abs/2305.17529.

Xijie Huang, Li Lyna Zhang, Kwang-Ting Cheng, and Mao Yang. 2023. Boosting llm reasoning: Push the limits of few-shot learning with reinforced in-context pruning. *ArXiv preprint*, abs/2312.08901.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023a. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore. Association for Computational Linguistics.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023b. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *ArXiv preprint*, abs/2310.06839.

Hoyoun Jung and Kyung-Joong Kim. 2023. Discrete prompt compression with reinforcement learning. *ArXiv preprint*, abs/2308.08758.

Byeongchang Kim, Hyunwoo Kim, and Gunhee Kim. 2019. Abstractive summarization of Reddit posts with multi-level memory networks. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2519–2531, Minneapolis, Minnesota. Association for Computational Linguistics.

Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*.

Mahnaz Koupaee and William Yang Wang. 2018. Wikihow: A large scale text summarization dataset. *ArXiv preprint*, abs/1810.09305.

Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, Singapore. Association for Computational Linguistics.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to compress prompts with gist tokens. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. 2023. Memgpt: Towards llms as operating systems. *ArXiv preprint*, abs/2310.08560.

Allen Roush and Arvind Balaji. 2020. DebateSum: A large-scale argument mining and summarization dataset. In *Proceedings of the 7th Workshop on Argument Mining*, pages 1–7, Online. Association for Computational Linguistics.

Uri Shaham, Maor Ivgi, Avia Efrat, Jonathan Berant, and Omer Levy. 2023. ZeroSCROLLS: A zero-shot benchmark for long text understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 7977–7989, Singapore. Association for Computational Linguistics.

Claude E Shannon. 1951. Prediction and entropy of printed english. *Bell system technical journal*, 30(1):50–64.

Kristina Toutanova, Chris Brockett, Ke M. Tran, and Saleema Amershi. 2016. A dataset and evaluation metrics for abstractive compression of sentences and short paragraphs. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 340–350, Austin, Texas. Association for Computational Linguistics.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

David Wingate, Mohammad Shoeybi, and Taylor Sorensen. 2022. Prompt compression and contrastive conditioning for controllability and toxicity reduction in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 5621–5634, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Zheng Zhao, Shay B. Cohen, and Bonnie Webber. 2020. Reducing quantity hallucinations in abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2237–2249, Online. Association for Computational Linguistics.

A Details of Data Distillation

To construct the extractive compression dataset, we use `GPT-4-32k` to compress the original meeting transcript. Each transcript is divided into chunks first, with each chunk terminating at the end of a complete sentence and not exceeding 512 tokens. We employ the default parameter settings with a temperature of 0.3 and a `top_p` of 1.0. The maximum number of generated tokens is set to 4096. Transcripts exceeding 28K tokens are truncated, allowing a 4K token budget for generation. Fig. 9 presents the full instruction used in GPT-4 compression. Tab. 8 shows the statistics of our MeetingBank compression dataset.

Data Part	Data Size	Chunk	Sentence (Avg)	Token (Avg)	$1/\tau$
Original	5,169	41,746	232	3,635	-
Compressed	5,169	41,746	132	1,415	2.57x

Table 8: Statistics of MeetingBank compression dataset.

B Details of Data Annotation

Based on the compressed prompt, we design a word annotation algorithm to automatically assign each word a label indicating whether the word in the original prompt should be retained. Initially, all labels of the original words are set to *False*. Then, for every word in the compressed prompt, we search for its corresponding word in the original prompt, which is then assigned a *True* label.

Sliding Window: To assign labels to the appropriate words in the original prompt, we utilize a sliding window approach, constraining the search scope within a local window centered on the previously matched word in the original prompt.

Prompt Compression Details:
Example 1:
Item 15, report from City Manager Recommendation to adopt three resolutions. First, to join the Victory Pace program. Second, to join the California first program. And number three, consenting to to inclusion of certain properties within the jurisdiction in the California Hero program. It was emotion, motion, a second and public comment. CNN. Please cast your vote. Oh. Was your public comment? Yeah. Please come forward. I thank you, Mr. Mayor. Thank you. Members of the council. My name is Alex Mitchell. I represent the hero program. Just wanted to let you know that the hero program. Has been in California for the last three and a half years. We're in. Over 20. We're in 28 counties, and we've completed over 29,000 energy efficient projects to make homes. Greener and more energy efficient. And this includes anything. From solar to water. Efficiency. We've done. Almost. $550 million in home improvements.
Example 2:
John: So, um, I've been thinking about the project, you know, and I believe we need to, uh, make some changes. I mean, we want the project to succeed, right? So, like, I think we should consider maybe revising the timeline.
Sarah: I totally agree, John. I mean, we have to be realistic, you know. The timeline is, like, too tight. You know what I mean? We should definitely extend it .

Figure 6: *LLMLingua-2* performs context awareness compression. The dark red highlights the words which are preserved at a 5x compression ratio, medium red denotes 3x compression ratio, and light red represents 2x compression ratio. Gray indicates discarded words during compression.

The search initiates from the last matching position. The *True* label is then assigned to the first matched word in the original prompt. Furthermore, the search is bidirectional to prevent mismatches caused by GPT-4's reordering, as shown in Fig. 5. Moreover, if GPT-4 introduces new words during compression, the sliding window restricts the search scope, preventing mismatches between the newly added words in the compressed prompt and words in the original prompt.

Fuzzy Matching: Another challenge arises from the "variation" misbehavior of GPT-4, as illustrated in Fig 5. GPT-4 may alter the original words in tense, voice, and singular/plural forms during compression, even when we request GPT-4 to compress by discarding words only. To address this issue, we first apply lemmatization to reduce words to their base form using Spacy[5], and then perform word matching using the sliding window approach.

C Context Aware Compression

Fig. 6 presents some compression results of our *LLMLingua-2* under different compression ratios. Our method effectively maintains the most meaningful words as the compression ratio increases.

D Comparison with Baselines

In Fig. 11 and Fig. 12, we qualitatively compare the compressed prompts of our methods with those of baseline method on GSM8K and BBH datasets. Note our *LLMLingua-2* here is only trained on MeetingBank, but also yields more reasonable compressed prompt than baseline methods on the transferred domain data.

E Prompt Reconstruction

Fig. 7 and Fig. 8 show two reconstructed prompts from the compressed prompts using GPT-4. Specifically, we prepend a simple reconstruction instruction: "*I have asked you to compress a meeting transcript by dropping word only. Now, reconstruct the original meeting transcript based on the following compressed transcript.*" to the compressed prompt. With the key information preserved in the compressed prompt, the reconstructed prompt closely resembles the original prompt.

F Details of MeetingBank QA and MeetingBank Summary

The MeetingBank QA dataset consists of 862 meeting transcripts from the MeetingBank test set. Initially, we generate 10 question-answer pairs for each meeting transcript using GPT-4-32K. The instruction used in generating QA pairs is: "*Create 10 questions/answer pairs from the given meeting transcript. The answer should be short and concise. The question should start with Q: and answser should start with A: . The meeting transcript is as follows.*". To ensure the quality of the generated QA pairs, we discard the question-answer pairs with answer lengths exceeding 50 tokens. Subsequently, we carefully examine the remaining QA pairs to ensure that the answers actually appear in the original transcripts, instead of being products of GPT-4's hallucinations. After the aforemen-

[5]https://spacy.io/api/lemmatizer

Original Prompt (200 Tokens):
Item 15, report from City Manager Recommendation to adopt three resolutions. First, to join the Victory Pace program. Second, to join the California first program. And number three, consenting to to inclusion of certain properties within the jurisdiction in the California Hero program. It was emotion, motion, a second and public comment. CNN. Please cast your vote. Oh. Was your public comment? Yeah. Please come forward. I thank you, Mr. Mayor. Thank you. Members of the council. My name is Alex Mitchell. I represent the hero program. Just wanted to let you know that the hero program. Has been in California for the last three and a half years. We're in. Over 20. We're in 28 counties, and we've completed over 29,000 energy efficient projects to make homes. Greener and more energy efficient. And this includes anything. From solar to water. Efficiency. We've done. Almost.$550 million in home improvements.
Compressed Prompt (98 Tokens, 2x Compression):
Item 15 report City Manager Recommendation adopt three resolutions. join Victory Pace program. Second join California first program. three inclusion properties jurisdiction California Hero program. emotion motion second public comment. CNN. cast vote. public comment?. come forward. thank you Mr. Mayor. Members council. Alex Mitchell. represent hero program. California three and a half years. 20. 28 counties completed 29,000 energy efficient projects. includes. solar water. Efficiency Almost $550 million home improvements.
Reconstructed prompt (198 Tokens):
Item number 15 on the agenda is the report from the City Manager. The Recommendation is to adopt three resolutions. The first is to join the Victory Pace program. The second resolution is to join the California First program. The third resolution is for the inclusion of properties within our jurisdiction into the California Hero program. There is an emotion for the motion and a second. We will now open the floor for public comment. We have a representative from CNN here. Are there any other individuals who would like to cast a vote or make a public comment? If so, please come forward.
Thank you, Mr. Mayor and Members of the council. My name is Alex Mitchell and I represent the Hero program in California. I have been with the program for three and a half years. As of now, 20 out of 28 counties have completed 29,000 energy efficient projects. This includes projects related to solar water and energy efficiency. Almost $550 million has been invested in home improvements.

Figure 7: Prompt reconstruction results of our *LLMLingua-2* on the MeetingBank example.

Original Prompt (160 Tokens):
Okay. Thank you. Next step is we're going to do item number, is it that was 16. So I could do item 16. We'll try to get through these as expeditiously as possible. And there's going to be a a motion that's ready to go here. So can we the the the item please. Report from city clerk recommendation to receive and file the certification of the petition regarding the regulation of medical marijuana businesses and approve one of the following three alternative actions adopt the initiative ordinance without alteration to submit the initiative ordinance without alteration to the voters to be held on November 8th, 2016 or three. Adopt a report pursuant to California State Elections Code. Thank you. There's a motion and a second device. Marie Lowenthal. Thank you.
Compressed Prompt (83 Tokens, 2x compression):
Okay. Thank you. Next step do item number, 16. item 16. try get through expeditiously. motion ready go here. item. Report from city clerk receive file certification of petition regulation medical marijuana businesses approve three alternative actions adopt initiative ordinance without alteration submit to voters November 8th, 2016 or three. Adopt report California State Elections Code. motion second device. Marie Lowenthal.
Reconstructed Prompt (123 Tokens):
Okay. Thank you. The next step is to do item number 16. For item 16, we will try to get through this expeditiously. The motion is ready to go from here. The item is a report from the city clerk to receive and file the certification of a petition for the regulation of medical marijuana businesses. We can approve three alternative actions: adopt the initiative ordinance without alteration, submit it to the voters on November 8th, 2016, or three, adopt the report as per the California State Elections Code. The motion is seconded by the device. Marie Lowenthal.

Figure 8: Prompt reconstruction results of our *LLMLingua-2* on the MeetingBank example.

tioned filtering process, we retain 3 high-quality question-answer pairs for each meeting transcript. Additionally, we instruct GPT-4-32K to summarize each meeting transcript. The summaries generated by GPT-4 are used as ground truth to evaluate the summary performance.

G Drawback of Existing Text Compression Dataset

Existing extractive compression datasets such as SentComp (Filippova and Altun, 2013) and DebateSum (Roush and Balaji, 2020) are mainly created for summarization task. The compressed texts provided in their dataset are usually too concise, only maintaining the main idea of the original text and lacking detailed information. This information loss inevitably hinders the downstream tasks such as document-based QA, as illustrated in Fig. 13 and Fig. 14

Our GPT-4 Instruction for Compression:
System Prompt:
You are an excellent linguist and very good at compressing passages into short expressions by removing unimportant words, while retaining as much information as possible.
User Prompt:
Compress the given text to short expressions, and such that you (GPT-4) can reconstruct it as close as possible to the original. Unlike the usual text compression, I need you to comply with the 5 conditions below:
1. You can ONLY remove unimportant words.
2. Do not reorder the original words.
3. Do not change the original words.
4. Do not use abbreviations or emojis.
5. Do not add new words or symbols.
Compress the origin aggressively by removing words only. Compress the origin as short as you can, while retaining as much information as possible. If you understand, please compress the following text: {*text to compress*}
The compressed text is:

Figure 9: The instruction we used in GPT-4 compression.

Instruction1:
Could you please rephrase the paragraph to make it short, and keep 5% tokens?
Instruction2:
Summarize the provided examples in a few sentences, maintaining all essential reasoning aspects.
Instruction3:
Remove redundancy and express the text concisely in English, ensuring that all key information and reasoning processes are preserved.
Instruction4:
Follow these steps to shorten the given text content: 1. First, calculate the amount of information contained in each sentence, and remove sentences with less information. 2. Next, further condense the text by removing stop words, unnecessary punctuation, and redundant expressions. Refine the content while ensuring that all key information is retained. Let's do it step by step.

Figure 10: Other instructions we evaluated, which are proposed in LLMLingua (Jiang et al., 2023a).

H Model Size and Training Details

We use `xlm-roberta-large` which has 355M parameters as the feature encoder f_θ in `LLMLingua-2`. The training process takes approximately 23 hours on our MeetingBank compression dataset. For `LLMLingua-2-small`, the feature encoder is the `multilingual-BERT` which has 110M parameters. It takes roughly 16 hours to train the `multilingual-BERT` model.

I GPU Memory Usage

LLMLingua-2 enjoys a smaller GPU memory overhead because of its lightweight. The peak GPU memory usage of LLMLingua-2 on MeetingBank is only 2.1GB, while LLMLingua and Selective-Context, which utilize LLAMA-2-7B as the SLM, consume 16.6GB and 26.5GB of GPU memory, respectively.

J Multilingual Generalization Ability

In Table 10, we assess the performance of *LLMLingua-2* on the Chinese benchmarks of LongBench, comprising 5 tasks with a total of 1000 samples. Despite being trained solely on the MeetingBank data, which consists of English corpus only, *LLMLingua-2* also outperforms *LLMLingua* on Chinese benchmarks. We attribute this performance gain to the multilingual capabilities of the `xlm-roberta-large` or `multilingual-BERT` compressor acquired from the pre-training phase.

K Integration with LongLLMLingua

In retrieval-augmented generation (RAG) and Multi-Documents Question-Answer (MDQA) scenarios, the primary challenge is to identify the document that contains the key information relevant to the question. In these scenarios, *LongLLMLingua* improves the key information preservation by utilizing the information provided in the question.

While *LLMLingua-2* is designed for question-agnostic compression, it can also be integrated with *LongLLMLingua* to preserve more key information relevant to the question in these scenarios. Specifically, we utilize *LongLLMLingua's* coarse-grained

Original Prompt (139 tokens):
Q: I have a blackberry, a clarinet, a nectarine, a plum, a strawberry, a banana, a flute, an orange, and a violin. How many fruits do I have?
A: Let's think step by step.
We first identify the fruits on the list and include their quantity in parentheses:
- blackberry (1) - nectarine (1) - plum (1) - strawberry (1) - banana (1) - orange (1)
Now, let's add the numbers in parentheses: 1 + 1 + 1 + 1 + 1 + 1 = 6. So the answer is 6.
Compressed prompt (57 tokens) by *LLMLingua*:
: a blackberry, a a ne a a a a, many have
:'s think
We first theruits the list and include their in - (–
's the numbers in parentheses:1 + 1 = 6. So the answer is 6.
Compressed prompt (54 tokens) by *LLMLingua-2*:
Q: clarinet, nectarine, strawberry, violin.
How many fruits
think step by step.
identify fruits include quantity parentheses:
blackberry nectarine plum strawberry banana orange add numbers parentheses: 1 + 1 = 6.
answer is 6.

Figure 11: Comparison with baseline. *LLMLingua-2* here is only trained on MeetingBank, but also yields more reasonable compressed prompt than *LLMLingua* on BBH.

compression to assign varying compression ratios to different documents based on the question's perplexity conditioned on each document. Consequently, it allocates more token budgets to the documents which are more relevant to the question.

As illustrated in Table 11, *LLMLingua-2* with *LongLLMLingua* coarse-grained compression achieves an average performance gain of 25.3% on NaturalQuestions (Liu et al., 2024) compared to *LLMLingua-2*.

L Sample-Wise Dynamic Compression Ratio

By default, *LLMLingua-2* applies fixed compression rate to all samples in the benchmark. However, this approach may not be optimal due to variations in the density of key information across different samples. To address this problem, we allow *LLMLingua-2* to dynamically adjust the compression rate for each sample under the overall compression rate constraint. Specifically, we employ the compressor to predict each token's preservation probability of all samples. We then set a probability threshold to achieve the overall compression rate constraint. For all samples, tokens with preservation probabilities higher than this threshold are retained.

Table 12 presents the performance of *LLMLingua-2* using the sample-wise dynamic compression ratio, showcasing a 4.4% and 4.5% performance improvement under 7x and 5x compression ratios, respectively, compared to *LLMLingua-2* with a fixed compression ratio.

M Performance *w.r.t* Compression Ratio

Fig 15 presents the performance *w.r.t* compression ratio on a subset of 100 samples from Meetingbank. As depicted, LLMLingua-2 exhibits superior robustness compared to other baselines as the compression ratio increases.

N Preservation Priority in GPT-4 Compression

To gain insight into GPT-4's compression patterns, we analyze the distribution of different POS categories. Fig 16 suggests that GPT-4 prioritizes the preservation of nouns, adjectives, and numerals, which typically play a more important role in the comprehension of the overall context.

O Comparison With GPT-4 Compression

Table 9 shows the comparison between LLMLingua-2 compressed prompts and GPT-4 compressed prompts. For GPT-4 compression, We use the same compression instruction as the one used in training data collection. The same chunking technique is also adopted with the chunk size setting to 512. It is shown that LLMLingua-2 achieves higher performance than GPT-4 compression on MeetingBank QA. We conjecture that LLMLingua-2's ability to learn compression knowledge from the entire dataset helps mitigate the influence of noise and information loss present

Original Prompt (249 tokens):
Question: Sam bought a dozen boxes, each with 30 highlighter pens inside, for $10 each box. He rearranged five of these boxes into packages of six highlighters each and sold them for $3 per package. He sold the rest of the highlighters separately at the rate of three pens for $2. How much profit did he make in total, in dollars?
Let's think step by step
Sam bought 12 boxes x $10 = $120 worth of highlighters.
He bought 12 * 30 = 360 highlighters in total.
Sam then took 5 boxes × 6 highlighters/box = 30 highlighters.
He sold these boxes for 5 * $3 = $15
After selling these 5 boxes there were 360 - 30 = 330 highlighters remaining.
These form 330 / 3 = 110 groups of three pens.
He sold each of these groups for $2 each, so made 110 * 2 = $220 from them.
In total, then, he earned $220 + $15 = $235.
Since his original cost was $120, he earned $235 - $120 = $115 in profit.
The answer is 115
Compressed prompt (144 tokens) by *LLMLingua*:
: Sam bought a dozen boxes each 30 highl pens inside, $10 each. He reanged five of boxes into of
six each $3 per. He sold the thelters separately at the of three $2. much make total,
Lets think step
bought boxes x0 oflters
He 2 3ters in
Sam then boxes 6lters/box 0ters
He sold these boxes 5
Afterelling these boxes there 36030lters
ese00 of three
sold groups2 each so made *2 $20 from
In total, he015
Since his he $ - $120 = $115 in profit.
The answer is 115
Compressed prompt (138 tokens) by *LLMLingua-2*:
Sam bought dozen 30 highlighter pens $10 rearranged five boxes into six highlighters sold $3 per sold rest three pens profit ?
Sam bought 12 boxes x $10 = $120
12 * 30 = 360 highlighters
5 boxes × 6 highlighters/box = 30
sold 5 * $3 = $15
5 360 - 30 = 330 highlighters
330 / 3 = 110 groups three
sold $2 110 * 2 = $220
earned $220 + $15 = $235. original cost earned $235 - $120 = $115
The answer is 115

Figure 12: Comparison with baseline. *LLMLingua-2* here is only trained on MeetingBank, but also yields more reasonable compressed prompt than *LLMLingua* on GSM8K.

Methods	QA	Length	
	EM	Tokens	$1/\tau$
GPT-4 Compression	84.86	1,221	2.5x
LLMLingua-2-small	85.82	984	3.0x
LLMLingua-2	**86.92**	970	3.1x
Original	87.75	3,003	1.0x

Table 9: Comparison with GPT-4 compressed prompt on MeetingBank.

in each GPT-4 compressed example, leading to superior performance.

P Performance of Mistral-7B on 8K Token Subset

As the Mistral 7B model is trained with an 8k context length [6] , its performance may drop if the input context is too long. Therefore, we conduct additional experiments on subsets containing only examples with original prompts shorter than 8k tokens. The results, shown in Table 13, demonstrate that LLMLingua-2 continues to outperform strong baselines and even the original prompts in this subset.

[6]https://huggingface.co/docs/transformers/main/en/model_doc/mistral

Document:
Chinese government is to open more museums, memorial halls and national patriotism education bases to the public for free amid efforts to upgrade cultural services.All national museums and provincial comprehensive museums will stop charging entry fees this year, says a government circular. Museums and memorial halls listed as national patriotism education bases will open for free, adds the circular, jointly issued by the Publicity Department of the Communist Party of China Central Committee, the ministries of finance and culture, and the State Administration of Cultural Heritage on Janyary 23. Free entry is also available to museums above county level in Zhejiang, Fujian, Hubei, Jiangxi, Anhui and Gansu provinces and Xinjiang Uygur Autonomous Region. Other provinces, autonomous regions and municipalities are encouraged cut or abolish entry fees according to their circumstances, the circular says. All museums, memorial halls and national patriotism education bases will be free to visit by 2009 except cultural relics and historical sites, which will have cheap rates for minors, the elderly, soldiers, the disabled and low-income families, says the circular. For special or guest exhibitions, museums and memorial halls can charge fees, the circular says, and museums are encouraged to have cheap tickets and flexible plans, such as regular free entry, and cheap tickets for groups and families.
Question:
In which provinces will museums above country level be open for free?

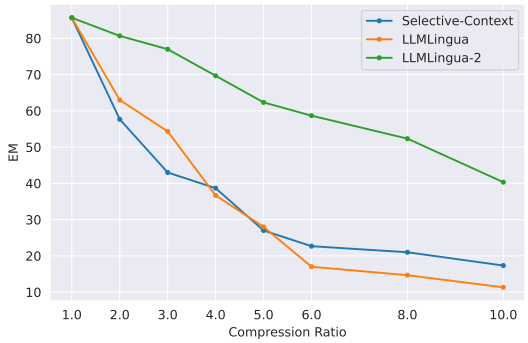
Figure 13: An example from the *SentComp* dataset (Filippova and Altun, 2013). The compressed text is highlighted in blue. The provided compressed text fails to cover the question references which are highlighted in red.

Document:
The overall results regarding the long-term effects of exchange rate volatility are highly informative in relation to the exports and imports of an LDC. Mexico's exports of agricultural goods are clearly depressed by uncertainty: Table 3 shows that no unprocessed agricultural good responds positively, while various animal, vegetable, and wood products make up 6 of the 21 industries with negative effects. Imports are also affected. While the category of Oil-seeds, oil nuts, and oil kernels does seem to increase because of uncertainty, 6 of the 21 industries in which volatility reduces import flows are agricultural in nature. Mexican textile exports also show clear negative effects due to uncertainty, not only for the category of Clothing except fur clothing, but also for the inputs of Textile and leather machinery and Textile yarn and thread (in Table 4).
Question:
Which industries of textile suffer from negative effects due to the exchange rate uncertainty?

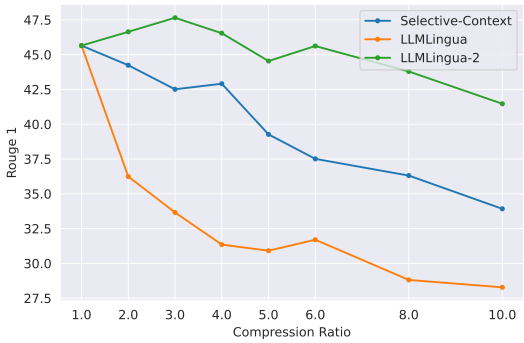
Figure 14: An example from the *DebateSum* dataset (Roush and Balaji, 2020). The compressed text is highlighted in blue. The provided compressed text fails to cover the question references which are highlighted in red.

Methods	LongBench-Zh							
	SingleDoc	MultiDoc	Summ.	FewShot	Synth.	**AVG**	Tokens	$1/\tau$
Task(Question)-Agnostic Compression								
LLMLingua	35.2	20.4	11.8	24.3	51.4	28.6	3,060	5x
LLMLingua-2	**46.7**	**23.0**	**15.3**	**32.8**	**72.6**	**38.1**	3,023	5x
Original Prompt	61.2	28.7	16.0	29.2	77.5	42.5	14,940	-

Table 10: Out-of-domain evaluation on LongBench **Chinese** benchmarks.



(a) QA performance *w.r.t* compression ratio on a 100 samples subset of MeetingBank.



(b) Summary performance *w.r.t* compression ratio on a 100 samples subset of MeetingBank.

Figure 15: A plot of performance *w.r.t* compression ratio on a 100 samples subset of MeetingBank.

Methods	1st	5th	10th	15th	20th	Reorder	Tokens	$1/\tau$
4x constraint								
Question-Aware Compression								
BM25†	40.6	38.6	38.2	37.4	36.6	36.3	798	3.7x
Gzip†	63.1	61.0	59.8	61.1	60.1	62.3	824	3.6x
SBERT†	66.9	61.1	59.0	61.2	60.3	64.4	808	3.6x
OpenAI†	63.8	64.6	65.4	64.1	63.7	63.7	804	3.7x
LLMLingua-2^{+}	74.0	70.4	67.0	66.9	65.3	71.9	739	3.9x
LongLLMLingua†	**75.0**	**71.8**	**71.2**	**71.2**	**74.7**	**75.5**	748	3.9x
Question-Agnostic Compression								
Selective-Context†	31.4	19.5	24.7	24.1	43.8	-	791	3.7x
LLMLingua†	25.5	27.5	23.5	26.5	30.0	27.0	775	3.8x
LLMLingua-2	48.6	44.5	43.6	40.9	39.9	46.2	748	3.9x
Original Prompt	75.7	57.3	54.1	55.4	63.1	-	2,946	-
Zero-shot	56.1						15	196x

Table 11: Performance comparison on NaturalQuestions (20 documents) (Liu et al., 2024). *LLMLingua-2*$^{+}$ denotes *LLMLingua-2* with *LongLLMLingua* (Jiang et al., 2023b) coarse level compression. †: numbers reported in Jiang et al. (2023b).

Methods	**LongBench-SingleDoc**					
	QA Score	Tokens	$1/\tau$	QA Score	Tokens	$1/\tau$
Target Token Constraint	*2,000 Tokens*			*3,000 Tokens*		
LLMLingua-2	29.8	1,954	7.4x	35.5	3,392	4.3x
Compression Ratio Constraint	*7x*			*5x*		
LLMLingua-2 FR†	25.1	2,131	6.8x	27.4	3,185	4.5x
LLMLingua-2 DCR‡	**29.5**	2,125	6.8x	**32.2**	3,164	4.5x
Original Prompt	39.7	14,511	1x	39.7	14,511	1x

Table 12: Evaluation of LLMLingua-2 sample wise dynamic compression on LongBench single doc QA task. FR† assigns each example with the same fixed compression rate. DCR‡ assigns dynamic compression rate to different examples within the corpus level constraint.

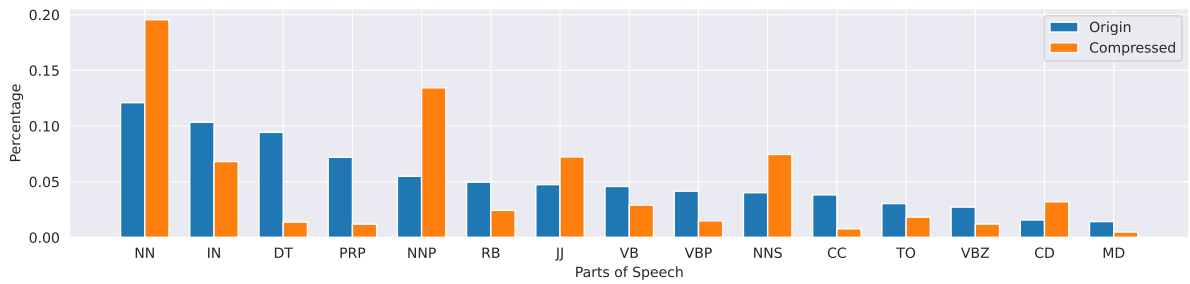


Figure 16: Part of speech distribution of the original prompts and GPT-4 compressed prompts.

Methods	MeetingBank				LongBench-SingleDoc					
	QA	Summ.	Tokens	$1/\tau$	2,000-token cons.	Tokens	$1/\tau$	3,000-token cons.	Tokens	$1/\tau$
Selective-Context	62.43	19.25	703	2.4x	29.3	1,829	2.5x	34.6	2,855	1.6x
LLMLingua	51.78	24.57	714	2.4x	29.9	1,862	2.5x	30.7	3,016	1.5x
LLMLingua-2	**81.75**	**30.83**	651	2.6x	**35.0**	1,889	2.4x	**36.3**	2,841	1.6x
Original Prompt	71.27	27.56	1,700	-	31.4	4,595	-	31.4	4,595	-

Table 13: Evaluation with Mistral-7B as the Target LLM on MeetingBank and LongBench single doc QA task. We discarded samples where the input text has more than 8K tokens. We report Rouge1(Lin, 2004) for summary.